Chapter 7

Structured Query Language

(SQL)

JKU, Dept. of Computer Science, By Mequanent S., 2018 E.C

December 4,2025

Structured Query Language (SQL)

Lesson 1

What is SQL?

- **SQL** is the standard language used to communicate with a relational database.
- SQL is the language you use to express your needs to the database.
- you can request specific information from the database in the form of a query using SQL.
- SQL is an ANSI standard.
- **ANSI,** *stands for American National Standards Institute, is an organization that is responsible* for devising standards for various products and concepts.

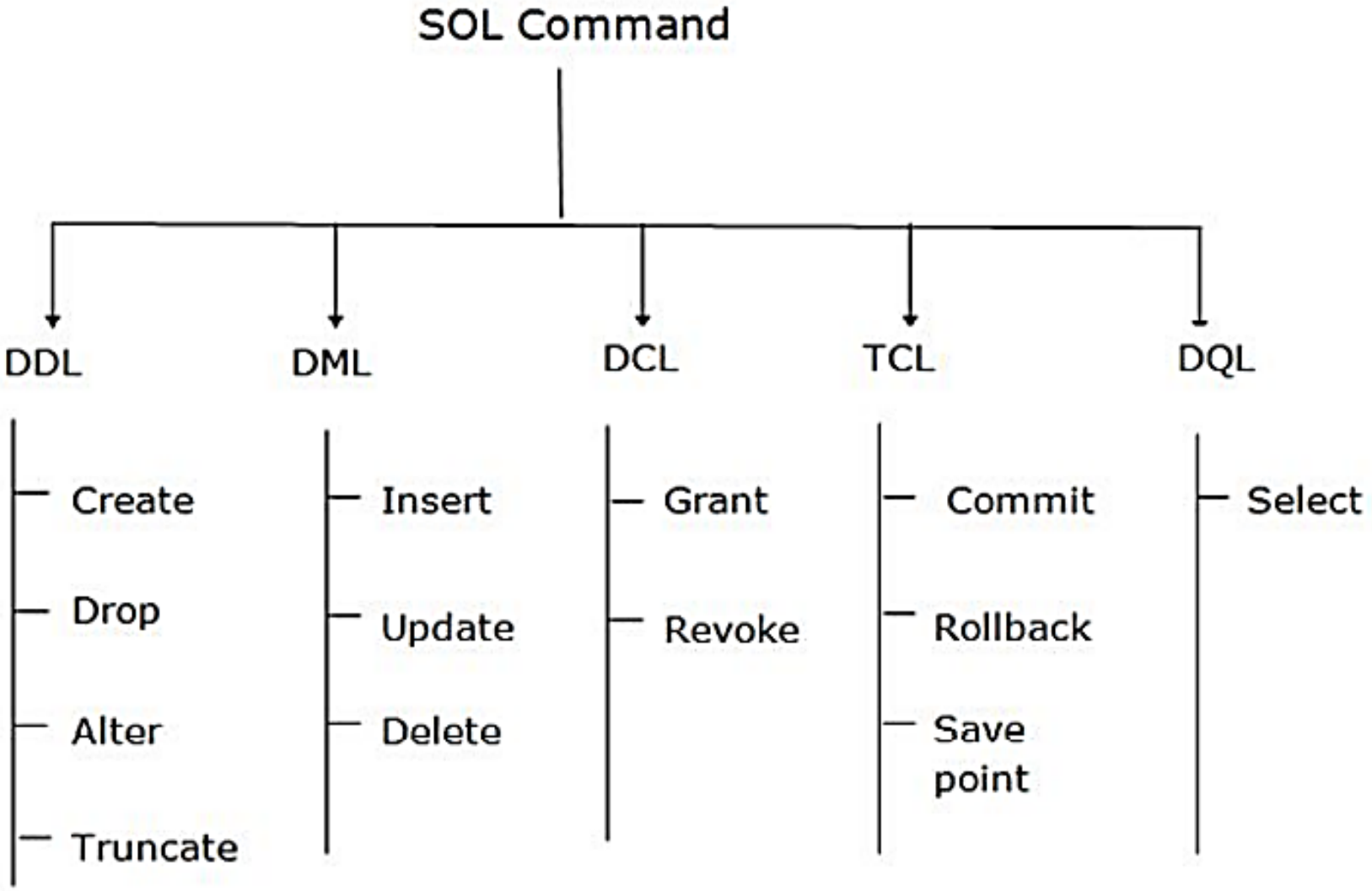
What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

Types of SQL Commands

- The basic categories of commands used in a SQL to perform various functions are:
  1. DDL (Data Definition Language)
  2. DML(Data Manipulation Language)
  3. DCL (Data Control Language)
  4. TCL (TransactionControl Language)
  5. DQL (Data Query Language)

Types of SQL Commands



1. Data Definition Language (DDL)

- *DDL, is the part of SQL that allows a database user to create and* restructure database objects, such as the **creation** or the **deletion** of a table.
- i.e. **Defining Database Structures**
- The main DDL commands include:
  - **CREATE**
  - **ALTER**
  - **DROP**

- The principal logical data definition statements are:
  - CREATE **DATABASE**,
  - CREATE **TABLE**,
  - ALTER **TABLE**,
  - DROP **TABLE**,
  - DROP **DATABASE**

2. Data Manipulation Language (DML):

- *DML, is the part of SQL used to manipulate data within objects of* a relational database.
- There are three basic DML commands:
  - **INSERT**
  - **UPDATE**
  - **DELETE**
  - **SELECT**

3. Data Control Language (DCL):

- DCL in SQL allow you to control access to data within the database.
- DCL is mainly related to the security issues,
  - i.e. who has access to the database objects and what operations they can perform on them.

- The task of DCL is to prevent unauthorized access to data.
- These DCL commands are normally used to create objects related to user access and also control the distribution of privileges among users.
- Some data control commands are as follows:
  - **ALTER PASSWORD**
  - **GRANT**
  - **REVOKE**
  - **CREATE SYNONYM**

4. Transaction Control Language (DCL):

- TCL in SQL help users to manage and control a database's transactions (changes) to maintain consistency.
- It also allows statements to be grouped together into logical transactions.
- TCL allows the statements to be grouped together into logical transactions.
- These TCL commands are normally used to manage the changes made by DML statements.
- Some transaction control commands are as follows:
  - **1. COMMIT**
  - **2. SAVEPOINT**
  - **3. ROLLBACK**
  - **4. SET TRANSACTION**

5. Data Query Language (DQL):

- DQL is the most commonly used SQL statements for a relational database user.
- This SQL statement enables the users to query one or more tables to get the information they want.
- SQL has only one data query statement:
  - **SELECT**

Lets start SQL query

- We will start our SQL query with the common SQL commands:
  - **CREATE**
  - **INSERT**
  - **SELECT**
  - **DELETE**
  - **UPDATE**
- Procedures to open **Microsoft SQL server**
  - Start -> All program -> Microsoft SQL Server 2008 -> click on SQL server management studio -> Select Server Type to "Database Engine" -> Select Server name -> Click Connect.

Creating and Dropping Database

- **The Create statement:**
  - **Syntax:**
  - **CREATE <object type> <object name>**
  - **Object type-** can be database or table
  - **Object name-** can be database name or table name
- ***CREATING DATABASE***
  - The most basic syntax for the CREATE DATABASE statement looks like :
  - CREATE DATABASE <*database name*>

Example:

Create database ***myFirstDB;***

- **The Drop statement:**
  - Used to remove the databases.
  - **Syntax:**
  - **Drop <object type> <object name>**
  - **Object type-** can be database or table
  - **Object name-** can be database name or table name
- ***Dropping Database***
  - The most basic syntax for the DROP DATABASE statement looks like :
  - DROP DATABASE <*database name*>

Example:

Drop database ***myFirstDB;***

Creating Tables

The 'CREATE TABLE' Statement

- used to create a table in a database.

Syntax:

```
Use Database_Name

CREATE TABLE table_name
    (
    column_name1 data_type,
    column_name2 data_type,
    column_name3 data_type,
    ....
    );
```

- The data type specifies what type of data the column can hold.

- Example:

```
use myFirstDB

create table Student
(
    stud_id varchar(10),
    stud_Fname varchar(50),
    stud_Lname varchar(50),
    stud_age integer,
    stud_sex varchar(50)
);
```

Adding Attributes

- **Syntax:**

  ALTER TABLE Table_Name

  ADD column_name data_type;

- The **ALTER TABLE** command allows the user to change the structure of a table.
- **Example:**

  ALTER Table student

  Add Stud_grade varchar(1);

Deleting Attributes

- **Syntax:**

  ALTER TABLE Table_Name

  DROP COLUMN column_name;

- **Example:**

  ALTER Table student

  DROP COLUMN Stud_age;

Deleting The Table

- Syntax:

```
DROP TABLE table_name;
```

- Example:

```
Drop table student;
```

Exercise

1. Create a database called JKU2.
2. Delete the database JKU2.
3. Re-create the database JKU2.
4. Create tables called DEPARTMENT and COURSE.

   Department (dept_ID, dept_Name, dept_Location).

   Course (C_Code, C_Title).

5. Add the attribute ***C_Description*** for course table.
6. Remove the attribute ***dept_Location*** from department table.

Answers

1. CREATE DATABASE JKU2;
2. DROP DATABASE JKU2;
3. USE JKU2;
4. CREATE TABLE Department (dept_ID int Primary key ,dept_Name varchar(10), dept_Location varchar(10));

   CREATE TABLE Course (C_Code int Primary key, C_Title varchar(10));
5. ALTER TABLE Course

   ADD C_Description varchar(10);
6. AlTER TABLE Departement

   DROP COLUMN dept_Location;

Inserting Values to the Table

- **Syntax:**

INSERT INTO table_Name (column 1, column 2, …) Values(value 1, value2, …);

- **Example:**

```
INSERT INTO student (stud_id, stud_Fname,
stud_Lname, stud_sex,)
VALUES ('201','Mr.X','Mr.Y','M');
```

Or

```
INSERT INTO student
VALUES (201,'Mr.X','Mr.Y','M');
```

Displaying Data

- **Syntax :**

```
Select * from Table_name;
```

- Example:

```
Select * from student;
```

End of the lesson-1!!!

SQL

Lesson 2

Lesson Topics

- SQL Data Types
- SQL Constraints
- SQL SELECT Statement

Lesson objectives

- **To know the different SQL Data types.**
- **To have a good understanding on SQL Constraints.**
- **To know how those constraints are implemented on the database.**
- **To understand how data is retrieved from the database.**

SQL Data types

- SQL data type is a characteristics that specifies type of data of any object.
- Each column, variable and expression has the same data type in SQL.
- You would use these data types while creating your tables.
- You would choose a particular data type for a table column based on your requirement.
- SQL Server offers the following categories of data types for your use:

Data types…

Character String Data Types:

Data Type	Description
CHARACTER (n), CHAR(n)	Fixed-length character string, where ***n*** is the number of characters.
CHARACTER VARYING(n), CHAR VARYING(n), VARCHAR (n)	Varying-length character string, where ***n*** is the maximum number of characters.
INTEGER, INT, SMALLINT	integer numbers of various sizes

SQL Constraints

- Constraints are the rules enforced on data columns on table.
- These are used to limit the type of data that can go into a table.
- This ensures the accuracy and reliability of the data in the database.
- Constraints could be column level or table level.
- Column level constraints are applied only to one column where as table level constraints are applied to the whole table.

Constraints...

- Following are commonly used **constraints** available in SQL:

1. NOT NULL Constraint

2. DEFAULT Constraint

3. UNIQUE Constraint

4. PRIMARY KEY

5. FOREIGN KEY

6. CHECK Constraint

Constraints…

1. NOT NULL Constraint:

- By default, a column can hold NULL values.
- If you do not want a column to have a NULL value then you need to define such constraint on this column specifying that NULL is now not allowed for that column.
  - i.e. it ensures that a column cannot have NULL value.
- A NULL is not the same as no data, rather, it represents unknown data.

Constraints…

- **Example:** the following SQL creates a new table called STUDENT and adds six columns, three of which, ID, NAME and AGE, specify not to accept NULLs:

```
CREATE TABLE STUDENT
(
    STUD_ID   INT  NOT NULL,
    STUD_NAME VARCHAR (20)    NOT NULL,
    STUD_AGE  INT  NOT NULL,
    STUD_ADDRESS  CHAR (25) ,
    STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2)
);
```

Constraints…

2. DEFAULT Constraint :

- This constraint provides a default value to a column when the INSERT INTO statement does not provide a specific value.
- **Example:**

```
CREATE TABLE STUDENT
(
    STUD_ID   INT                  NOT NULL,
    STUD_NAME VARCHAR (20)     NOT NULL,
    STUD_AGE  INT                  NOT NULL,
    STUD_ADDRESS  CHAR (25) ,
     STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2) DEFAULT 3.50,
 );
```

- Here STUD_GPA column is set to 3.50 by default, so in case INSERT INTO statement does not provide a value for this column then by default this column would be set to 3.50 .

Constraints…

3. UNIQUE Constraint :

- This Constraint prevents two records from having identical values in a particular column.
  - i.e. Ensures that all values in a column are different
- **Example:**

```
CREATE TABLE STUDENT
(
    STUD_ID   INT  NOT NULL  UNIQUE,
    STUD_NAME VARCHAR (20)     NOT NULL,
    STUD_AGE  INT  NOT NULL,
    STUD_ADDRESS  CHAR (25) ,
     STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2) DEFAULT 3.50,
);
```

- Here STUD_ID column is set to UNIQUE, so that you can not have two records with same STUD_ID.

Constraints…

4. PRIMARY KEY (PK):

- A PK is a field in a table which uniquely identifies each rows/records in a database table.
- Primary keys must contain unique values.
- A primary key column cannot have NULL values.
- A table can have only one PK which may consist of single or multiple fields.
- When multiple fields are used as a PK, they are called a **composite key**.
- If a table has a PK defined on any field(s) then you can not have two records having the same value of that field(s).

Constraints…

- **Example:**
- Creating a table STUDENT with a PK of student Identification Number.

```
CREATE TABLE STUDENT
(
    STUD_ID   INT   NOT NULL,
    STUD_NAME VARCHAR (20)    NOT NULL,
    STUD_AGE  INT  NOT NULL,
    STUD_ADDRESS  CHAR (25) ,
    STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2) DEFAULT 3.50,
    PRIMARY KEY (STUD_ID)
);
```

Constraints…

- To create a PK constraint on the “STUD_ID" column when STUDENT table already exists:
  - ALTER TABLE STUDENT ADD PRIMARY KEY (STUD_ID);
- For defining a PK constraint on multiple columns:

```
CREATE TABLE STUDENT
(
    STUD_ID   INT    NOT NULL,
    STUD_NAME VARCHAR (20)     NOT NULL,
    STUD_AGE  INT                  NOT NULL,
    STUD_ADDRESS  CHAR (25) ,
     STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2) DEFAULT 3.50,
     PRIMARY KEY (STUD_ID, STUD_NAME)
 );
```

Constraints…

5. FOREIGN KEY (FK):

- A FK is a key used to link two tables together. This is sometimes called a **Referencing key.**
- A FK is a field in a table which uniquely identifies each rows/records in any another database table.
- FK is a column or a combination of columns whose values match a PK in a different table.
- The relationship between two tables matches the PK in one of the table with a FK in the second table

Constraints…

Example:

- Consider the following two tables: STUDENT and DEPARTMENT

```
CREATE TABLE STUDENT
(
    STUD_ID   INT,
    STUD_NAME VARCHAR (20),
    STUD_AGE  INT,
    STUD_ADDRESS  CHAR (25) ,
     STUD_SEX CHAR (5),
    STUD_GPA   DECIMAL (3, 2),
     DEPT_ID INT REFERENCES DEPARTMENT(DEPT_ID),
     PRIMARY KEY (STUD_ID)
 );

CREATE TABLE DEPARTMENT
(
    DEPT_ID   INT,
    DEPT_NAME VARCHAR (20),
    DEPT_LOCATION VARCHAR (20),
     PRIMARY KEY (DEPT_ID)
 );
```

Constraints…

- If DEPARTMENT table has already been created, and the foreign key has not yet been, use the syntax for specifying a foreign key by altering a table.

```
ALTER TABLE DEPARTMENT
ADD FOREIGN KEY (STUD_ID) REFERENCES
  STUDENT(STUD_ID);
```

Constraints…

6. CHECK Constraint:

- It enables a condition to check the value being entered into a record.
  - i.e. It ensures that all values in a column satisfy certain conditions.

- If the condition evaluates to false, the record violates the constraint and isn't entered into the table.

Constraints…

- **Example:**

```
CREATE TABLE STUDENT
(
      STUD_ID   INT,
      STUD_NAME VARCHAR (20),
      STUD_AGE  INT NOT NULL CHECK (STUD_AGE >= 18),
      STUD_ADDRESS  CHAR (25) ,
       STUD_SEX CHAR (5),
      STUD_GPA   DECIMAL (3, 2),
      PRIMARY KEY (STUD_ID)
 );
```

- Here we add a CHECK with STUD_AGE column, so that you cannot have any student below 18 years.

SQL SELECT STATEMENT

- used to retrieve data or information from table (database).
- used to fetch the data from a database table which returns data in the form of result table.
- **Syntax:**

  SELECT <attribute list> **FROM** <table list>

  - **<attribute list>** is a list of attribute names whose values are to be retrieved by the query.
  - **<table list>** is a list of the relation names required to process the query.

SELECT…

- **Example:** to fetch only ID, First name and Sex of Employee from employee table:
  - SELECT Emp_ID,Emp_Fname,Emp_Sex FROM EMPLOYEE
- If you want to fetch **all the fields** available in the table then you can use following syntax:
  - SELECT * FROM table_name;
- **Example:** **to** fetch **all fields available in the table DEPARTMENT**
  - SELECT * FROM DEPARTMENT;
- The Astrix **(*)** symbol stands for ***all the attributes***.

SQL DISTINCT keyword

- To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used.
- **For example**, the result of Q1 may have duplicate SALARY values where as Q2 does not have any duplicate values.

```
Q1:     SELECT   Emp_Salary
        FROM     EMPLOYEE

Q2:     SELECT   DISTINCT Emp_Salary
        FROM     EMPLOYEE
```

Exercise

Create two tables called DEPARTMENT and EMPLOYEE

DEPARTMENT(**Dept_ID**, Dept_Name, Dept_Location)

EMPLOYEE(**Emp_ID**, Emp_Fname, Emp_Lname, Emp_Sex, Emp_Age, Emp_Salary, ***Dept_ID***)

Note:

- **Bold** and **underline** attribute (s)refers that it is a **primary key.**
- ***bold*** and ***italics*** attribute(s) refers that it is a **foreign key.**

Make sure that:

- The Employee salary, first name and Department name should ***not allow*** a null value.
- The employee **age** should not be null and less than 18 years.
- Set a **default value** for the Department name to "**computer science**".

And finally insert at least 4 records of data to both tables.

END OF THE LESSON-2!!!

SQL

Lesson 3

Lesson Topics

1. SQL WHERE Clause
2. SQL UPDATE Statement
3. SQL DELETE Statement

SQL *WHERE* Clause

- The SQL **WHERE** clause is used to specify a condition while fetching the data from single table or joining with multiple tables.
- If the given condition is satisfied then only it returns specific value from the table.
- You would use WHERE clause to filter the records and fetching only necessary records.
- The WHERE clause not only used in SELECT statement, but it is also used in UPDATE, DELETE statement.

WHERE...

- **Syntax:**
- The basic syntax of SELECT statement with WHERE clause is as follows:

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition]
```

- You can specify a condition using comparison or logical operators like >, <, =, LIKE, NOT etc.

WHERE…

- **Example:** STUDENT relation:

Stud_ID	Stud_Fname	Stud_Lname	Stud_Sex	Stud_Age	Stud_GPA
101	Amare	Eshetu	M	30	3.75
102	Amir	Ahmed	M	27	3.25
103	Hana	Bedaso	F	25	3.00
104	Eftu	Abrahim	F	23	3.50
105	Eyasu	Wana	M	26	3.30

- **Q1:** Retrieve all female students from student relation.

```
SELECT *
FROM STUDENT
WHERE Stud_Sex='F';
```

WHERE…

- **Q2:** Retrieve all students whose age is above 25 years old from student relation.

```
SELECT *
FROM STUDENT
WHERE stud_Age > 25;
```

- **Q3:** Fetch students ID, First name, Sex and GPA fields from Student table for a student with GPA of 3.50.

```
SELECT Stud_ID,Stud_Fname,stud_Sex,Stud_GPA
FROM STUDENT
WHERE stud_GPA =3.50;
```

SQL *AND* and *OR* clause

- The SQL **AND** and **OR** operators are used to compile multiple conditions to narrow data in an SQL statement.
- These two operators are called **conjunctive operators.**
- These operators provide a means to make multiple comparisons with different operators in the same SQL statement.

The AND Operator:

- The **AND** operator allows the existence of multiple conditions in an SQL statement's **WHERE** clause.
- **Syntax:**
- The basic syntax of AND operator with WHERE clause is as follows:

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition1] AND [condition2]...AND [conditionN];
```

- You can combine **N** number of conditions using AND operator.
- For an action to be taken by the SQL statement, whether it be a transaction or query, all conditions separated by the AND must be **TRUE**.

And…

- **Example:**
- **Q1:** Retrieve all students from student Relation where students age is less than 27 and whose GPA is above 3.00.

```
SELECT *
FROM STUDENT
WHERE stud_Age < 27 AND Stud_GPA >3.00;
```

The OR Operator:

- The **OR** operator is used to combine multiple conditions in an SQL statement's **WHERE** clause.
- **Syntax:**
- The basic syntax of OR operator with WHERE clause is as follows:

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition1] OR [condition2]...OR [conditionN]
```

- You can combine **N** number of conditions using OR operator.
- For an action to be taken by the SQL statement, whether it be a transaction or query, only any ONE of the conditions separated by the OR must be **TRUE**.

or

- **Example:**
- **Q1:** Retrieve all students from student Relation where students **age** is less than 27 **or** whose **GPA** is above 3.00.

```
SELECT *
FROM STUDENT
WHERE stud_Age < 27 OR Stud_GPA >3.00;
```

SQL Update query

- The SQL **UPDATE** Query is used to modify the existing records in a table.
- You can use **WHERE** clause with **UPDATE** query to update selected rows otherwise all the rows would be effected.
- **Syntax:**
- The basic syntax of UPDATE query with WHERE clause is as follows:

```
UPDATE table_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

- You can combine **N** number of conditions using **AND** or **OR** operators.

Update…

- An additional **SET**-clause specifies the attributes to be modified and their new values.
- **Example:**
- Change the age of the student whose ID is 104 to 30.

```
UPDATE Student
SET Stud_Age=30
WHERE stud_ID =104;
```

- Change the First name and GPA of Student ID 102 to ‘Kebede' and 3.00, respectively.

```
UPDATE Student
SET Stud_Fname =‘Kebede’ ,Stud_GPA = 3.00
WHERE stud_ID =102;
```

Update…

- If you want to modify all GPA column values in STUDENT table, you do not need to use WHERE clause.

  i.e. :

- Change all GPA column of Student table to 3.85.

```
UPDATE STUDENT
SET stud_GPA =3.85;
```

SQL Delete query

- The SQL **DELETE** Query is used to delete the existing records from a table.
- You can use WHERE clause with DELETE query to delete selected rows, otherwise all the records would be deleted.

- **Syntax:**
- The basic syntax of DELETE query with WHERE clause is as follows:

```
DELETE FROM table_name
WHERE [condition];
```

- You can combine **N** number of conditions using **AND** or **OR** operators.

Delete…

- **Example:**
- Delete a student record of whose ID is 103.

```
DELETE FROM Student
WHERE stud_ID=103;
```

- Delete a record of a student either whose ID is 102 or whose Sex is Female.

```
DELETE FROM Student
WHERE stud_ID=102 OR Stud_Sex='F' ;
```

Exercise

- Create the following two tables:
- **STUDENT** Relation:

Stud_ID	Stud_Fname	Stud_Lname	Stud_Sex	Stud_Age	Stud_GPA	Dept_ID
101	Amare	Eshetu	M	30	3.75	1
102	Amir	Ahmed	M	27	3.25	4
103	Hana	Bedaso	F	25	3.00	2
104	Eftu	Abrahim	F	23	3.50	1
105	Eyasu	Wana	M	26	3.30	2
106	Almaz	Mekonen	F	27	3.75	1

- **DEPARTMENT** Relation:

Dept_ID	Dept_Name	Dept_Location
1	Computer Science	B1R20
2	Biology	B5R19
3	Chemistry	B5R20
4	Civil Engineering	B1R22
5	Industrial Engineering	B1R21

Exercise…

- **Q1:** select all computer science department students.
- **Q2:** select all female computer science department students.
- **Q3:** change the department of a student ID 106 to Biology.
- **Q4:** Delete the students record where whose Sex is Male and whose GPA is Greater than 3.50
- **Q5:** Retrieve all department names which is not assigned to the students.

END OF THE LESSON-3!!!

SQL

Lesson 4

Lesson topics

- SQL- Operators
- SQL - Aggregate Functions
- SQL – ORDER BY clause
- SQL – GROUP BY Clause

SQL Logical Operators:

- Here is a list of all the logical operators available in SQL.

Operator	Description
ALL	-used to compare a value to all values in another value set.
AND	-allows the existence of multiple conditions in an SQL statement's WHERE clause.
ANY	-used to compare a value to any applicable value in the list according to the condition.
BETWEEN --- AND	-used to search for values that are within a set of values, given the minimum value and the maximum value. -column value is between two values, including the end values specified in the range.
EXISTS	-used to search for the presence of a row in a specified table that meets certain criteria.

SQL Logical Operators: Cont…

Operator	Description
IN	-used to compare a value to a list of literal values that have been specified.
LIKE	-used to compare a value to similar values using wildcard operators (%). -used to list all rows in a table whose column values match a specified pattern.
NOT	It reverses the meaning of the logical operator with which it is used. Eg. NOT EXISTS, NOT BETWEEN, NOT IN etc. This is negate operator.
OR	-used to combine multiple conditions in an SQL statement's WHERE clause.
IS NULL	-used to compare a value with a NULL value. -used to display all the rows for columns that do not have a value.

EXAMPLE

- To Retrieve all students whose first name begins with a letter 'A':

```
SELECT * FROM STUDENT
WHERE Stud_Fname LIKE 'A%';
```

- To find the full name of the students with their age who have an age of between 23 and 27 years

```
SELECT Stud_Fname, Stud_Lname , Stud_Age
FROM STUDENT
WHERE Stud_Age BETWEEN 23 and 27 ;
```

- To find the names of students whose GPA is either 3.75 or 3.33 or 3.50:

```
SELECT Stud_Fname, Stud_Lname , Stud_GPA
FROM STUDENT
WHERE Stud_GPA IN (3.75,3.33,3.50)
```

EXAMPLE CONT…

- To find the names of students whose age is not specified in the student table:

```
SELECT Stud_Fname, Stud_Lname
FROM STUDENT
WHERE Stud_Age IS NULL;
```

- To display all students whose sex is either ***Male*** or ***Female*** and whose last name is either ***Ahmed*** or ***Bedaso*** or ***Kebede*** or whose first name starts with letter E and whose ID is not 105.

```
SELECT * FROM Student
WHERE (Stud_Sex =‘ M’ OR Stud_Sex =‘ F’) AND
(Stud_LName IN (‘Ahmed’, ‘Bedaso’, ‘Kebede’) OR
Stud_Fname LIKE ‘E%’ ) AND ID != 105;
```

Example: cont…

```
SELECT Stud_Age FROM STUDENT
WHERE EXISTS (SELECT Stud_Age FROM STUDENT
WHERE Stud_GPA > 3.50);

SELECT * FROM STUDENT
WHERE Stud_Age > ALL (SELECT Stud_Age FROM
      STUDENT WHERE Stud_GPA > 3.50);

SELECT * FROM STUDENT
WHERE Stud_Age > ANY (SELECT Stud_Age FROM
      STUDENT WHERE Stud_GPA > 3.50);
```

SQL Aggregate (Group) Functions

- Group functions are built-in SQL functions.
- They operate on groups of rows and return one value for the entire group.
- They return a single value, using values in a table column
- These functions are: **COUNT, MAX, MIN, AVG, SUM.**

SQL Aggregate (Group) Functions

Functions	Description
COUNT ()	• Returns the number of rows in the table that satisfies the condition specified in the WHERE condition. • If the WHERE condition is not specified, then the query returns the total number of rows in the table.
SUM()	• used to get the sum of a numeric column
AVG()	•used to get the average value of a numeric column.
MAX()	•used to get the maximum value from a column.
MIN()	•used to get the minimum value from a column.

EXAMPLE:

- If you want to get the total number of students in all of the departments, the query would take the form:

```
SELECT COUNT (*) FROM STUDENT;
```

- If you want to get the number of students in a particular department, the query would be:

```
SELECT COUNT (*) FROM STUDENT
WHERE Dept_ID = 1;
```

- To get the total salary given out to the employees,

```
SELECT SUM (Emp_Salary) FROM EMPLOYEE;
```

EXAMPLE CONT…

- To get the average of Student GPA:

  SELECT AVG (Stud_GPA) FROM STUDENT;

- To get the maximum of Student GPA:

  SELECT MAX (Stud_GPA) FROM STUDENT;

- To get the minimum of Student GPA:

  SELECT MIN (Stud_GPA) FROM STUDENT;

- change the student's of GPA to 3.67 where whose GPA is less than the average of GPA of the students:

```
UPDATE STUDENT SET Stud_GPA = 3.67
WHERE Stud_GPA < (SELECT AVG (Stud_GPA) FROM STUDENT);
```

SQL Statement	Description
ORDER BY	➢ used to order the data sets retrieved from a SQL database. ➢ You can use ORDER BY to order the retrieved data by more than one column. ➢ You can use ASC and DESC SQL keywords
GROUP BY	➢ used along with the group functions to retrieve data grouped according to one or more columns.
HAVING	➢ used to filter data based on the group functions. ➢ This is similar to WHERE condition but is used with group functions. ➢ Group functions cannot be used in WHERE Clause but can be used in HAVING clause.

EXAMPLE:

- If we want to sort our Student table by the First Name column,

```
SELECT *            FROM Student
ORDER BY Stud_Fname
```

- if you want to order by both Last Name and age columns,

```
SELECT * FROM Student
ORDER BY Stud_Lname, Stud_Age
```

- If you want to order descending by both columns (i.e. last name and age)

```
SELECT * FROM Student
ORDER BY Stud_Lname DESC, Stud_Age DESC
```

- But the default is ASC .

EXAMPLE CONT…

- **EMPLOYEE** Relation:

Emp_ID	Emp_Fname	Emp_Lname	Emp_Sex	Emp_Age	Emp_Dept	Emp_Salary
E101	Amare	Eshetu	M	30	Finance	2800
E102	Amir	Ahmed	M	27	HRM	3000
E103	Hana	Bedaso	F	25	HRM	3400
E104	Eftu	Abrahim	F	23	Finance	2500
E105	Eyasu	Wana	M	26	Library	2200

- If you want to know the total amount of salary spent on each department:

```
SELECT Emp_Dept, SUM (Emp_Salary)
FROM EMPLOYEE
GROUP BY Emp_Dept;
```

	Emp_Dept	(No column name)
1	Finance	5300.00
2	HRM	6400.00
3	Library	2200.00

EXAMPLE CONT…

- The **group by** clause should contain all the columns in the select list expect those used along with the group functions.

```
SELECT Emp_sex, Emp_Dept, SUM (Emp_Salary)
FROM EMPLOYEE
GROUP BY Emp_sex, Emp_Dept;
```

Output: (GROUP BY Emp_Dept, Emp_sex;)

	Emp_sex	Emp_Dept	(No column name)
1	F	Finance	2500.00
2	M	Finance	2800.00
3	F	HRM	3400.00
4	M	HRM	3000.00
5	M	Library	2200.00

	Emp_sex	Emp_Dept	(No column name)
1	F	Finance	2500.00
2	F	HRM	3400.00
3	M	Finance	2800.00
4	M	HRM	3000.00
5	M	Library	2200.00

EXAMPLE CONT…

- If you want to select the department that has total salary paid for its employees more than 5000:

```
SELECT Emp_Dept, SUM (Emp_Salary)
FROM EMPLOYEE
GROUP BY Emp_Dept
HAVING SUM (Emp_Salary) > 5000 ;
```

- **OUTPUT:**

	Emp_Dept	(No column name)
1	Finance	5300.00
2	HRM	6400.00

EXAMPLE CONT…

- When ***WHERE***, ***GROUP BY*** and ***HAVING*** clauses are used together in a **SELECT** statement:
  - the **WHERE** clause is **processed first**,
  - **then** the rows that are returned after the WHERE clause is executed are grouped based on the **GROUP BY** clause.
  - Finally, any conditions on the group functions in the **HAVING** clause are applied to the grouped rows before the final output is displayed.

END OF THE LESSON-4!!!

SQL

Lesson 5

Lesson topics

SQL LIKE clause

SQL TOP clause

SQL Like clauses

- The SQL **LIKE** clause is used to compare a value to similar values using **wildcard** operators.
- There are two **wildcards** used in conjunction with the **LIKE** operator:

  The percent sign (%)

  The underscore (_)
- The **percent sign** represents zero, one, or multiple characters.
- The **underscore** represents a single number or character.
- The symbols can be used in combinations.

Example:

- Here are number of examples showing WHERE part having different **LIKE** clause with '**%**' and '_' operators:

Statement	Description
WHERE SALARY LIKE '200%'	Finds any values that start with 200
WHERE SALARY LIKE '%200%'	Finds any values that have 200 in any position
WHERE SALARY LIKE '_00%'	Finds any values that have 00 in the second and third positions

Example cont…

Statement	Description
WHERE SALARY LIKE '%2'	Finds any values that end with 2
WHERE SALARY LIKE '_2%3'	Finds any values that have a 2 in the second position and end with a 3
WHERE SALARY LIKE '2___3'	Finds any values in a five-digit number that start with 2 and end with 3

SQL TOP clauses

- The SQL **TOP** clause is used to fetch a TOP **N** number records from a table.
- **Syntax:** SELECT TOP **number** C_Name FROM table_name
- **Example:**
- To get top 3 records from Employee table:

```
SELECT TOP 3 * FROM EMPLOYEE
```

- To get top 3 records from Employee table based on their salary:

```
SELECT TOP 3 * FROM EMPLOYEE
ORDER BY Emp_salary DESC;
```

END OF THE LESSON-5!!!

SQL

Lesson 6

Lesson Topics

- SQL sp_rename
- SQL Union
- SQL Intersect
- Join

Example:

EMPLOYEE

Emp_ID	Emp_Fname	Emp_Lname	Emp_Sex	Emp_Age	Emp_Salary	Dept_ID
E101	Amare	Eshetu	M	30	2800	D105
E102	Amir	Ahmed	M	27	3000	D102
E103	Hana	Bedaso	F	25	3400	D103
E104	Eftu	Abrahim	F	23	2500	D102
E105	Eyasu	Wana	M	26	2200	D103
E106	Almaz	Mekonen	F	27	3800	D105
E107	Werku	Alemu	M	35	5000	D105

DEPARTMENT

Dept_ID	Dept_Name	Dept_Location
D101	Finance	B1R4
D102	HRM	B1R5
D103	Library	B1R6
D104	Civil Engineering	B2R1
D105	Computer Science	B2R2

Renaming

- Synatx: to rename the relation's name:

  exec sp_rename old_Name, new_name;
- Example:
- To change the name of the relation EMPLOYEE to EMP:

  exec sp_rename EMPLOYEE, EMP;
- Syntax: to rename the attribute's name:

  exec sp_rename 'Relation_name.old_Name', 'new_name';
- Example: to change the column name of 'Emp_age' to 'Age' from Employee relation:

  exec sp_rename 'Employee.Emp_age', 'Age';

SQL Union

- The purpose of the SQL **UNION** query is to combine the results of two queries together.

- One of the restriction of **UNION** is that all **corresponding columns** need to be of the **same data type.**

- when using **UNION**, only distinct values are selected (similar to **SELECT DISTINCT**).

- **Syntax:**

[SQL Statement 1]
UNION
[SQL Statement 2]

Union Cont...

- Example:

```
SELECT Dept_ID FROM EMPLOYEE
union
SELECT Dept_ID FROM DEPARTMENT;
```

- You can select all values using UNION ALL :
- Example:

```
SELECT Dept_ID FROM EMPLOYEE
union all
SELECT Dept_ID FROM DEPARTMENT;
```

- What will be the output of the following SQL query:

```
SELECT Dept_ID, Emp_Salary FROM EMPLOYEE
union
SELECT Dept_ID, Emp_Name FROM DEPARTMENT;
```

SQL Intersect

- Similar to the **UNION** command, **INTERSECT** also operates on two SQL statements.

- The difference is that, while **UNION** essentially **acts as** an **OR** operator (value is selected if it appears in **either** the first or the second statement), the **INTERSECT** command **acts as** an **AND** operator (value is selected only if it appears in **both** statements).

- **Syntax :**

  [SQL Statement 1]
  INTERSECT
  [SQL Statement 2]

Intersect cont…

- Example:

  SELECT Dept_ID **FROM** EMPLOYEE

  intersect

  SELECT Dept_ID **FROM** DEPARTMENT;

- One of the restriction of **INTERSECT** is that all corresponding columns need to be of the same data type.

- And also All queries combined using a UNION, INTERSECT or EXCEPT operator must have an **equal number of attributes**.

Cross join (Cartesian product)

- A cross join is a join of tables **without specifying** the **join condition**.
- In this scenario, the query would return **all possible combination** of the tables in the SQL query.
- **Example:**

```
SELECT * FROM EMPLOYEE, DEPARTMENT;
```

Theta join:

- It is a join of tables with **specifying** the join condition.
- **Example:**

```
SELECT *
FROM EMPLOYEE e join DEPARTMENT d on
e.Dept_ID = d.Dept_ID;
```

Left outer join

- In a left outer join, all rows from the first table mentioned in the SQL query is selected, regardless whether there is a matching row on the second table mentioned in the SQL query.
- Example

  SELECT * **FROM** EMPLOYEE e left outer join DEPARTMENT d **ON** e.DEPT_ID = d.DEPT_ID ;

Result:

	Emp_ID	Emp_Fname	Emp_Lname	Emp_Sex	Emp_Age	Emp_Salary	DEPT_ID	DEPT_ID	DEPT_NAME	DEPT_LOCATION
1	E101	Amare	Eshetu	M	30	2800.00	D105	D105	Computer Science	B2R2
2	E102	Amir	Ahmed	M	27	3000.00	D102	D102	HRM	B1R5
3	E103	Hana	Bedaso	F	25	3400.00	D103	D103	Library	B1R6
4	E104	Eftu	Abrahim	F	23	2500.00	D102	D102	HRM	B1R5
5	E105	Eyasu	Wana	M	26	2200.00	D103	D103	Library	B1R6
6	E106	Almaz	Mekonen	F	27	3800.00	D106	NULL	NULL	NULL
7	E107	Werku	Alemu	M	35	5000.00	D105	D105	Computer Science	B2R2

right outer join

- In a Right outer join, all rows from the second table mentioned in the SQL query is selected, regardless whether there is a matching row on the first table mentioned in the SQL query.
- Example:

  SELECT * **FROM** EMPLOYEE e right outer join DEPARTMENT d **ON** e.DEPT_ID = d.DEPT_ID ;
- **Result:**

	Emp_ID	Emp_Fname	Emp_Lname	Emp_Sex	Emp_Age	Emp_Salary	DEPT_ID	DEPT_ID	DEPT_NAME	DEPT_LOCATION
1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	D101	Finance	B1R4
2	E102	Amir	Ahmed	M	27	3000.00	D102	D102	HRM	B1R5
3	E104	Eftu	Abrahim	F	23	2500.00	D102	D102	HRM	B1R5
4	E103	Hana	Bedaso	F	25	3400.00	D103	D103	Library	B1R6
5	E105	Eyasu	Wana	M	26	2200.00	D103	D103	Library	B1R6
6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	D104	Civil Engineering	B2R1
7	E101	Amare	Eshetu	M	30	2800.00	D105	D105	Computer Science	B2R2
8	E107	Werku	Alemu	M	35	5000.00	D105	D105	Computer Science	B2R2

full outer join

- In a Full outer join, all rows from the first and second table mentioned in the SQL query are selected, padding rows with NULL when no matching rows are found.
- Example

```
SELECT * FROM EMPLOYEE e Full outer join DEPARTMENT d
ON e.DEPT_ID = d.DEPT_ID ;
```

Result:

	Emp_ID	Emp_Fname	Emp_Lname	Emp_Sex	Emp_Age	Emp_Salary	DEPT_ID	DEPT_ID	DEPT_NAME	DEPT_LOCATION
1	E101	Amare	Eshetu	M	30	2800.00	D105	D105	Computer Science	B2R2
2	E102	Amir	Ahmed	M	27	3000.00	D102	D102	HRM	B1R5
3	E103	Hana	Bedaso	F	25	3400.00	D103	D103	Library	B1R6
4	E104	Eftu	Abrahim	F	23	2500.00	D102	D102	HRM	B1R5
5	E105	Eyasu	Wana	M	26	2200.00	D103	D103	Library	B1R6
6	E106	Almaz	Mekonen	F	27	3800.00	D106	NULL	NULL	NULL
7	E107	Werku	Alemu	M	35	5000.00	D105	D105	Computer Science	B2R2
8	NULL	NULL	NULL	NULL	NULL	NULL	NULL	D101	Finance	B1R4
9	NULL	NULL	NULL	NULL	NULL	NULL	NULL	D104	Civil Engineering	B2R1

END OF THE LESSON-6!!!